

MindAgent

前言

有需要的可以下载pdf版本。但是请不要倒卖，这个项目我沉淀了很久，定价也不是很高，感谢🙏（最后有用的话记得多推荐一下😊）

求反馈（重要 ）

大家觉得这个项目怎么样？用过这个项目的或者看完项目的可以在小红书找我给我反馈一下用着怎么样（球球了，想收集一波建议，也找一下优化方向）。

本文档从业务视角说明 MindAgent 如何处理一次客服请求：什么时候切换 Agent，什么时候并行协作，什么时候压缩会话，什么时候更新用户画像，什么时候触发升级或降级。

重要说明: MindAgent 当前业务链路已经支持端到端评测。除了 /chat 处理真实用户会话外，系统还可以通过 POST /eval/run 模拟业务问题，真实调用 Agent 编排链路生成回复，再用 LLM-as-Judge 对回复质量打分，并输出意图识别准确率、回归检测结果和优化建议。因此本项目不是只能“能聊”，还可以持续评估 Agent 在业务场景中的表现。

1. 一次用户请求的完整流程

用户调用 /chat 或在 CLI 输入消息后，系统按下面顺序处理：

代码块

- ```

1 用户消息
2 -> 读取记忆上下文
3 - Redis 当前会话最近消息
4 - ChromaDB 相关历史摘要
5 - ChromaDB 用户画像
6 -> 意图识别
7 - LLM 语义理解
8 - Embedding 相似度
9 - Pattern 关键词兜底
10 -> 知识库检索
11 - 查询改写
12 - 并行召回 ChromaDB knowledge_base
13 - LLM 重排 Top-K

```

```
14 -> Agent 路由
15 - 普通问题 -> GeneralAgent
16 - 技术问题 -> TechnicalAgent
17 - 账单/账户问题 -> BillingAgent
18 - 转人工/紧急问题 -> 升级标记
19 -> Skills 匹配与注入
20 - 按用户消息关键词匹配 Skills
21 - 按 Agent 类型过滤 general / technical / billing
22 - 将匹配到的处理规范注入 system prompt
23 -> Agent 基于记忆 + 知识库 + Skills 生成回复
24 -> 写入工作记忆
25 -> 异步更新用户画像
26 -> 返回给用户
```

评测调用 /eval/run 时，系统按下面顺序处理：

#### 代码块

```
1 评测用例
2 -> 意图识别评测
3 - 对比 predicted_intent 和 expected_intent
4 - 计算 Accuracy / Macro-F1
5 -> 端到端对话评测
6 - 调用 AgentOrchestrator.run()
7 - 单轮或多轮生成真实 Agent 回复
8 -> LLM-as-Judge
9 - 相关性 relevance
10 - 准确性 accuracy
11 - 完整性 completeness
12 - 有用性 helpfulness
13 -> 回归检测
14 - 与历史 baseline 对比
15 - 输出 regressions 和 recommendations
```

## 2. 什么时候会切换 Agent

MindAgent 并不是固定使用一个 Agent，而是根据用户问题动态选择。

### 2.1 技术问题切到 TechnicalAgent

当意图识别结果是 technical，或者问题中包含明显技术关键词时，会切换到技术 Agent。

典型输入：

#### 代码块

- 1 应用一直报 401 错误
- 2 网页打不开
- 3 系统崩溃了
- 4 出现 500 error
- 5 无法登录

常见关键词：

代码块

- 1 崩溃、报错、error、crash、无法登录、登录失败、500、401

业务效果：

代码块

- 1 GeneralAgent -> TechnicalAgent

TechnicalAgent 会更倾向于给出排障步骤、错误原因和下一步处理建议。当前 TechnicalAgent 还会匹配 skills/technical\_support/[SKILL.md](#)，在回复中遵循故障信息收集、错误码排查、部署配置检查、安全边界和二线升级规则。

## 2.2 账单/退款/账户问题切到 BillingAgent

当意图识别结果是 billing 或 account，或者问题包含账单关键词时，会切换到账单 Agent。

典型输入：

代码块

- 1 我要退款
- 2 为什么重复扣款了？
- 3 帮我开发票
- 4 我要取消订阅
- 5 修改我的邮箱

常见关键词：

代码块

- 1 退款、扣款、发票、账单、支付、订阅、refund、invoice

业务效果：

代码块

```
1 GeneralAgent -> BillingAgent
```

BillingAgent 会更关注退款、发票、订阅、账户资料等问题。当前 BillingAgent 还会匹配 [skills/billing\\_support/SKILL.md](#)，在回复中遵循账单核验、退款审核、发票处理和敏感信息保护等规则。

## 2.3 普通咨询使用 GeneralAgent

如果问题没有明确落到技术或账单领域，会默认使用 GeneralAgent。

典型输入：

代码块

```
1 你好
2 你们支持哪些功能？
3 订单什么时候到？
4 怎么联系客服？
```

业务效果：

代码块

```
1 GeneralAgent
```

GeneralAgent 负责通用问答和简单引导。当前 GeneralAgent 还会匹配 [skills/general\\_customer\\_service/SKILL.md](#)，在回复中遵循首轮接待、信息澄清、问题分流、投诉升级和禁止索取敏感信息等规则。

## 2.4 转人工或高紧急度会触发升级

如果用户明确要求转人工，或者问题被识别为紧急，会触发升级标记。

典型输入：

代码块

```
1 我要转人工
2 我要投诉
3 找你们经理
4 非常紧急，马上处理
```

业务效果：

代码块

```
1 escalated = true
```

当前项目里升级是标记状态；生产系统中可以在这个位置接入工单系统、人工客服队列或告警通知。

### 3. 什么时候会多 Agent 并行协作

如果一句话同时涉及多个业务领域，系统会自动并行调用多个 Agent。

#### 3.1 技术 + 账单复合问题

典型输入：

代码块

```
1 登录报错 401，而且这个月还重复扣款了
```

系统会识别到：

代码块

- ```
1   技术问题：登录报错 401
2   账单问题：重复扣款
```

业务处理：

代码块

```
1   TechnicalAgent + BillingAgent 并行处理
```

返回形式类似：

代码块

- ```
1 [technical]
2 针对 401 登录错误的排查步骤...
3
4 [billing]
5 针对重复扣款的处理建议...
```

### 3.2 为什么要并行而不是只选一个 Agent

如果只选 TechnicalAgent，账单问题可能被忽略；如果只选 BillingAgent，登录问题可能无法解决。  
并行协作的价值：

|               |                    |
|---------------|--------------------|
| 业务问题          | 并行协作的收益            |
| 一个问题跨多个领域     | 避免漏答               |
| 用户一次性提出多个诉求   | 分领域处理，更完整          |
| 专属 Agent 能力不同 | 每个 Agent 回答自己擅长的部分 |

### 4. 什么时候会降级到 GeneralAgent

降级是系统可靠性策略。即使专属 Agent 出问题，也尽量给用户一个通用答复。  
会降级的情况：

|                     |                  |
|---------------------|------------------|
| 情况                  | 结果               |
| 目标 Agent 类型没有可用实例   | 使用 GeneralAgent  |
| TechnicalAgent 执行失败 | 降级到 GeneralAgent |
| BillingAgent 执行失败   | 降级到 GeneralAgent |
| LLM 调用异常            | 返回通用错误提示         |

业务效果：

代码块

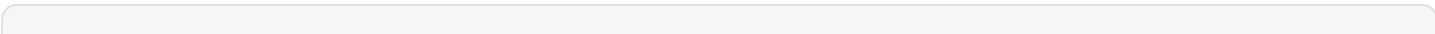
```
1 专属 Agent 失败 -> GeneralAgent 兜底
```

这样用户不会直接看到系统崩溃，而是收到“稍后重试”或通用处理建议。

### 5. 什么时候会话会被压缩

MindAgent 会把当前会话最近消息存在 Redis 中，叫工作记忆。

默认规则：



```
1 WORKING_MAX = 20
2 COMPRESS_AT = 15
```

当同一个 user\_id + conv\_id 的工作记忆达到 15 条消息时，会触发压缩。

## 5.1 压缩前

Redis 中可能有很多轮消息：

代码块

```
1 user: 你好
2 assistant: 你好，请问有什么可以帮你？
3 user: 我要退款
4 assistant: 请提供订单号
5 ...
```

## 5.2 压缩时

系统会：

代码块

```
1 旧消息 -> LLM 总结 -> 会话摘要
```

然后执行三件事：

代码块

```
1 1. 摘要写入 Redis summary:{user_id}:{conv_id}
2 2. 摘要写入 ChromaDB episodic
3 3. Redis 工作记忆只保留最近 5 条
```

## 5.3 压缩后

后续对话上下文由三部分组成：

代码块

```
1 会话摘要
2 相关历史
3 最近 5 条消息
```

业务价值：

|         |                   |
|---------|-------------------|
| 问题      | 压缩带来的收益           |
| 对话太长    | 控制 prompt 长度      |
| 历史信息不能丢 | 用摘要保留关键信息         |
| 后续还要查历史 | 写入 ChromaDB 供语义检索 |

## 6. 什么时候会更新用户画像

每次 /chat 回复完成后，系统都会异步更新用户画像。

触发时机：

代码块

```
1 Agent 回复完成 -> 写入本轮对话 -> 后台更新用户画像
```

画像内容示例：

代码块

```
1 {
2 "preferences": ["喜欢简洁回答", "经常咨询退款"],
3 "entities": {
4 "产品": ["会员服务"],
5 "问题类型": ["账单", "登录"]
6 }
7 }
```

业务作用：

|        |                  |
|--------|------------------|
| 用户画像信息 | 后续作用             |
| 偏好     | 让回复更贴近用户习惯       |
| 常见产品   | 帮助 Agent 理解用户上下文 |
| 常见问题类型 | 帮助后续回复更聚焦        |

注意：用户画像更新是异步执行的，不会阻塞当前回复。



## 7. 什么时候会查询知识库

知识库现在有两种使用方式：

| 入口      | 用途                                   |
|---------|--------------------------------------|
| /chat   | 主对话链路自动检索知识库，并把 Top-K 结果拼入 Agent 上下文 |
| /search | 单独演示和调试查询改写、并行召回、重排效果                |

典型业务问题：

代码块

```
1 退款多久到账？
2 订单超过 7 天还没到怎么办？
3 会员积分怎么算？
4 登录 401 怎么处理？
```

检索流程：

代码块

```
1 用户查询
2 -> LLM 查询改写
3 -> 多个子查询并行检索 ChromaDB knowledge_base
4 -> 合并去重
5 -> LLM 重排
6 -> 返回 Top-K
7 -> 拼入 Agent 背景信息
```

当知识库不可用时，会走 fallback，返回可解释的降级结果，而不是直接报错。

### 7.1 接入 /chat 后业务上有什么变化

接入前：

代码块

```
1 /chat 主要依赖 LLM 常识 + 对话记忆
2 /search 单独展示知识库检索
```

接入后：

代码块

```
1 /chat 会自动检索 knowledge_base
2 Agent 回复会优先参考知识库检索结果
```

例如用户问：

代码块

```
1 退款多久到账？
```

系统会先检索 knowledge\_base 中的退款政策，再把类似下面的内容放进 Agent 上下文：

代码块

```
1 [知识库检索结果]
2 1. 标题：退款政策
3 内容：退款申请提交后，系统会在 1-3 个工作日内审核。审核通过后，款项将在 5-7 个工作日内退回原支付账户。
```

Agent 再基于这段内容回复，能减少模型凭空编业务规则的风险。

## 8. 什么时候会加载和注入 Skills

Skills 是可热加载的业务处理规范，和知识库互补：

| 类型     | 解决的问题    | 示例                  |
|--------|----------|---------------------|
| 知识库    | 业务事实是什么  | 退款政策、配送说明、会员规则      |
| Skills | 客服应该怎么处理 | 如何澄清信息、何时转人工、哪些话不能说 |

启动时，api/main.py 会创建 SkillManager，从 MindAgent \_SKILLS\_DIR 指向的目录加载 Skills。默认目录是项目内 ./skills。

当前内置三类 Skills：

| Skill | 适用 Agent | 触发方式 |
|-------|----------|------|
|       |          |      |

|                          |                |                                |
|--------------------------|----------------|--------------------------------|
| general_customer_service | GeneralAgent   | 命中咨询、售后、客服、投诉、转人工等关键词          |
| technical_support        | TechnicalAgent | 命中报错、接口、API、部署、日志、500、401 等关键词 |
| billing_support          | BillingAgent   | 命中退款、扣款、发票、订阅、支付、账单等关键词        |

注入流程：

代码块

```
1 用户消息
2 -> Orchestrator 选择 Agent
3 - GeneralAgent.agent_type.value = "general"
4 - TechnicalAgent.agent_type.value = "technical"
5 - BillingAgent.agent_type.value = "billing"
6 -> BaseAgent._build_system_prompt()
7 -> SkillManager.prompt_for(message, agent_type)
8 -> 遍历已加载的 Skills
9 -> 筛选 enabled=true 的 Skill
10 -> 检查 Skill front matter 中的 agents 是否包含当前 agent_type
11 -> 检查用户消息是否命中 keywords
12 -> 拼入 system prompt 的 [动态 Skills] 区块
```

8.1 通用 Agent 加载 Skills 的具体例子

如果用户消息是：

代码块

```
1 你好，我想咨询售后处理进度
```

系统先识别为普通咨询，路由到 GeneralAgent。随后执行：

代码块

```
1 GeneralAgent
2 -> agent_type.value = "general"
3 -> BaseAgent._build_system_prompt(req)
4 -> SkillManager.prompt_for("你好，我想咨询售后处理进度", "general")
```

SkillManager 会检查 skills/general\_customer\_service/SKILL.md 顶部的配置：

代码块

```
1 agents: general
2 keywords: 你好,您好,咨询,帮助,客服,订单,售后,活动,会员,账户,资料,投诉,建议,人工,转人工,
 处理进度,服务
3 enabled: true
```

这个 Skill 同时满足三个条件：

| 条件                     | 当前结果 |
|------------------------|------|
| enabled=true           | 满足   |
| agents 包含 general      | 满足   |
| 用户消息命中 你好、咨询、售后 或 处理进度 | 满足   |

因此 通用客服接待规范 会被拼入 GeneralAgent 的 system prompt。最终 LLM 看到的系统提示由两部分组成：

代码块

```
1 GeneralAgent 原始 system_prompt
2
3 [动态 Skills]
4 通用客服接待规范
5 ...
```

如果同一条消息路由到 TechnicalAgent，由于 general\_customer\_service 的 agents 不包含 technical，它不会被注入。这个设计用来避免通用客服规则、技术排障规则和账单退款规则互相污染。

如果修改了 Skill 文件，可以调用：

代码块

```
1 curl -X POST http://localhost:8000/skills/reload
```

系统会重新扫描 Skill 目录，不需要重启服务。

查看当前加载状态：

代码块

```
1 curl http://localhost:8000/skills
```

返回内容会包含 Skill 名称、描述、文件路径、关键词、适用 Agent、是否启用和解析错误列表。

## 9. 什么时候 Monitor 会影响路由

Monitor 每 10 秒采集一次 Agent 和工具表现。

采集指标：

代码块

```
1 Agent 成功率
2 Agent 平均延迟
3 工具成功率
4 工具平均延迟
5 工具连续失败次数
```

如果某个 Agent 成功率低或延迟高，Monitor 会计算 monitor\_penalty 并写回 Orchestrator。

路由分数：

代码块

```
1 base_score = success_rate * 0.7 + latency_score * 0.3
2 routing_score = base_score * (1 - monitor_penalty)
```

业务效果：

代码块

```
1 表现差的 Agent -> routing_score 降低 -> 后续更少被选中
```

如果同一类 Agent 未来扩展成多个实例，这个机制可以自动选择表现更好的实例。

## 10. 常见业务场景示例

### 场景一：普通问候

代码块

```
1 用户：你好
```

- 2 Agent: GeneralAgent
- 3 结果: 普通欢迎语

## 场景二：技术故障

代码块

- 1 用户: 应用登录一直报 401
- 2 Agent: TechnicalAgent
- 3 Skills: 注入技术支持处理规范
- 4 结果: 给出登录失败排查步骤, 并提醒不要公开 Token、密码或验证码

## 场景三：退款问题

代码块

- 1 用户: 我要退款, 为什么还没到账?
- 2 Agent: BillingAgent
- 3 结果: 说明退款审核和到账周期

## 场景四：技术 + 账单复合问题

代码块

- 1 用户: 登录报错 401, 而且还重复扣款了
- 2 Agent: TechnicalAgent + BillingAgent
- 3 结果: 两个 Agent 并行回复后合并

## 场景五：账单 Skill 约束退款承诺

代码块

- 1 用户: 我要退款, 今天必须到账
- 2 Agent: BillingAgent
- 3 Skills: 注入账单退款处理规范
- 4 结果: 说明需要先核验订单和支付渠道, 不承诺“今天一定到账”

## 场景六：长对话

代码块

- 1 用户连续对话超过 15 条消息

- 2

系统：触发工作记忆压缩
- 3

结果：旧消息变成摘要，最近 5 条继续保留

场景七：强烈投诉

- 代码块
- 1

用户：我要投诉，马上转人工
- 2

系统：标记 `escalated = true`
- 3

结果：生产环境可转工单或人工客服

11. 业务规则速查表

| 触发条件                       | 系统行为                                       |
|----------------------------|--------------------------------------------|
| 普通咨询                       | 使用 GeneralAgent                            |
| 技术关键词或 technical 意图        | 使用 TechnicalAgent                          |
| 退款/扣款/发票/订阅/账户问题           | 使用 BillingAgent                            |
| GeneralAgent 处理普通咨询        | 注入通用客服接待规范 Skill                           |
| TechnicalAgent 处理技术问题      | 注入技术支持处理规范 Skill                           |
| BillingAgent 处理账单问题        | 注入账单退款处理规范 Skill                           |
| 修改 Skill 文件                | 调用 <code>/skills/reload</code> 热加载         |
| 技术 + 账单同时出现                | 多 Agent 并行协作                               |
| 专属 Agent 失败                | 降级到 GeneralAgent                           |
| 用户要求转人工                    | 标记 <code>escalated = true</code>           |
| 工作记忆达到 15 条                | 压缩旧消息，保留最近 5 条                             |
| 每次回复完成                     | 异步更新用户画像                                   |
| Agent 成功率低或延迟高             | Monitor 写回降权，降低 <code>routing_score</code> |
| 用户通过 <code>/chat</code> 提问 | 自动检索知识库并拼入 Agent 上下文                       |

知识库不可用

MCP 工具返回 fallback 降级  
结果，主对话继续运行